```
6. Create side-by-side comparisons of loss landscapes at different stages
of training for each operation. 7. Analyze how loss landscape metrics
change before, during, and after grokking. 8. Compare loss landscape
evolution between operations that grok quickly vs. slowly. 9. Investigate
correlations between changes in loss landscape metrics and the onset of
grokking.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Idea 27/50 - neural_collapse_grokking

```
"Name": "neural_collapse_grokking",
"Title": "Neural Collapse in Grokking: Investigating Feature Geometry
During Algorithmic Learning",
"Experiment": "1. Modify Transformer to output final layer features. 2.
Implement functions to compute class means and covariances. 3. Calculate
simplified neural collapse metrics: (a) average cosine similarity between
class means, (b) ratio of within-class to between-class variances. 4. Track
these metrics every 500 steps during training. 5. Run experiments on
modular arithmetic and permutation datasets. 6. Plot neural collapse
metrics alongside grokking curves. 7. Analyze changes in metrics before,
during, and after grokking. 8. Compare neural collapse dynamics between
operations that grok quickly vs. slowly. 9. Visualize class mean
trajectories in 2D/3D using PCA. 10. Discuss implications for understanding
both grokking and general neural network learning dynamics.",
"Interestingness": 9,
"Feasibility": 6,
"Novelty": 9,
"novel": true
```

Idea 28/50 - data_augmentation_grokking

```
"Name": "data_augmentation_grokking",
"Title": "Data Augmentation in Grokking: The Impact of Input
Transformations on Algorithmic Learning",
"Experiment": "1. Implement task-specific augmentations: (a) For modular
arithmetic: add random offsets (mod p) to inputs. (b) For permutations:
apply random permutations to inputs and outputs. 2. Modify GroupDataset to
apply augmentations with 0%
for each augmentation level across all datasets. 4. Track metrics: time to
grokking (99%
'augmentation generalization gap' (difference between augmented and non-
augmented validation accuracy). 5. Plot learning curves and generalization
gaps for each augmentation level. 6. Analyze the correlation between
augmentation level and grokking speed. 7. Compare attention patterns
between augmentation levels to understand representation changes. 8.
Discuss implications for designing data augmentation strategies in
algorithmic learning tasks.",
"Interestingness": 9,
"Feasibility": 9,
"Novelty": 8,
"novel": true
```

Idea 29/50 - emergent_grokking


```
"Name": "emergent_grokking",
"Title": "Emergent Abilities in Grokking: Investigating Scale-Dependent
Algorithmic Learning",
"Experiment": "1. Modify existing datasets to include 'simple' and
'complex' versions (e.g., mod sum with small vs. large primes). 2. Adjust
Transformer class to scale from tiny (1 layer, 64 dim) to medium (4 layers,
512 dim). 3. For each operation, train models of increasing size, tracking
grokking time and performance on both simple and complex versions. 4.
Implement a generalization test for each operation (e.g., mod sum with even
larger primes). 5. Plot learning curves for different model sizes,
highlighting grokking points. 6. Create heatmaps of model size vs.
operation complexity, showing grokking time and generalization test
results. 7. Perform statistical analysis to identify significant jumps in
performance across model sizes, using metrics such as accuracy increase
rate and time to reach 99%
behavior patterns across different operation types.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```


Idea 30/50 - functional_modularity_grokking


```
"Name": "functional_modularity_grokking",
"Title": "Functional Modularity in Grokking: Analyzing Emergent
Specialization in Transformer Networks During Algorithmic Learning",
"Experiment": "1. Implement functions to track weight update patterns and
attention focus for each layer and head. 2. Modify the training loop to
compute and store these metrics at regular intervals. 3. Define a
'functional modularity score' based on the consistency of weight updates
and attention patterns for specific input types. 4. Run experiments across
all datasets, tracking the functional modularity score alongside existing
metrics. 5. Plot the evolution of functional modularity alongside the
grokking curve. 6. Analyze how functional modularity changes before,
during, and after grokking. 7. Visualize the most consistent patterns at
different stages of training and interpret their functions. 8. Compare
functional modularity dynamics between different operations and model
sizes. 9. Investigate correlations between functional modularity and
grokking speed or generalization performance.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 9,
"novel": true
```


Idea 31/50 - information_compression_grokking


```
"Name": "information_compression_grokking",
"Title": "Information Compression in Grokking: Analyzing Representational
Dynamics During Algorithmic Learning",
"Experiment": "1. Modify Transformer class to include a bottleneck layer
(smaller dimension linear layer) after the encoder. 2. Implement function
to compute activation sparsity (%
```

```
bottleneck layer. 3. Update training loop to compute and store activation
sparsity and gradient magnitudes of the bottleneck layer at regular
intervals. 4. Run experiments with different bottleneck sizes (e.g., 25%
50%
to grokking, final validation accuracy, activation sparsity, and gradient
magnitudes. 6. Plot activation sparsity and gradient magnitude evolution
alongside grokking curves for each bottleneck size. 7. Analyze how these
metrics change before, during, and after grokking. 8. Test generalization
by evaluating models on slightly out-of-distribution examples (e.g., larger
numbers in modular arithmetic). 9. Investigate correlation between optimal
compression (measured by activation sparsity) and grokking speed,
generalization performance.",
"Interestingness": 9,
"Feasibility": 8,
"Novelty": 8,
"novel": true
```

Idea 32/50 - critical_learning_periods_grokking

```
"Name": "critical_learning_periods_grokking",
"Title": "Critical Learning Periods in Grokking: Temporal Dynamics of
Algorithmic Understanding",
"Experiment": "1. Modify the training loop to support 'intervention
periods' where learning rate is increased by 5x for 100 steps. 2. Implement
a sliding window intervention strategy, with windows of 500 steps, starting
every 250 steps. 3. Run experiments for each window across all datasets and
three model sizes (small, medium, large), including a control group with no
interventions. 4. Track metrics: time to grokking, final validation
accuracy, and 'intervention impact' (area under the validation accuracy
curve for 500 steps post-intervention). 5. Plot learning curves
highlighting intervention windows and their impacts. 6. Create heatmaps
visualizing intervention impact across time windows and model sizes for
each operation. 7. Analyze how intervention timing affects grokking across
different operations and model sizes. 8. Compare attention patterns
immediately before and after impactful interventions. 9. Investigate
whether certain operations or model sizes have more pronounced critical
periods than others. 10. Discuss implications for curriculum design in
machine learning and potential applications in continual and transfer
learning.",
"Interestingness": 9,
"Feasibility": 7,
"Novelty": 9,
"novel": true
```

Idea 33/50 - simplicity_bias_grokking

```
"Name": "simplicity_bias_grokking",
"Title": "Simplicity Bias in Grokking: Analyzing Weight Matrix Complexity
During Algorithmic Learning",
"Experiment": "1. Modify AbstractDataset to include two complexity levels
for each operation (e.g., small vs. large prime for modular arithmetic,
short vs. long permutations). 2. Implement a function to compute the
effective rank of weight matrices using singular value decomposition. 3.
Update the training loop to compute and store the effective rank for each
```